

Лекция 4

Массивы

1 Понятие массива

Массив — это непрерывный участок памяти, содержащий последовательность объектов одинакового типа, обозначаемый одним именем.

Элемент массива (значение элемента массива) — значение, хранящееся в определенной ячейке памяти, расположенной в пределах массива.

Адрес массива — адрес начального элемента массива.

Имя массива — идентификатор, используемый для обращения к элементам массива.

Размер массива — количество элементов массива.

Размер элемента — количество байт, занимаемых одним элементом массива.

Длина массива — количество байт, отводимое в памяти для хранения всех элементов массива.

Адрес	n	$n+k$	$n+2k$		$n+k(q-1)$	
Значение	$a[0]$	$a[1]$	$a[2]$	...	$a[q-1]$	
Индекс	0	1	2		$q-1$	

2 Объявление массива

Основная форма объявления массива размерности N:

тип <имя массива>[размер1][размер2]...[размерN]

Одномерный массив:

тип <имя массива>[размер];

Пример:

int a[100]; // целочисленный массив из 100 элементов (a[0]...a[99])

Двумерный массив (матрица):

тип <имя массива>[размер1][размер2];

Пример:

float b[2][3]; // вещественный массив (матрица) из 2 строк и 3 столбцов

float b[2][3]; // массив из 2 строк и 3 столбцов

Логическая структура:

	0	1	2
0	A[0][0]	A[0][1]	A[0][2]
1	A[1][0]	A[1][1]	A[1][2]

Представление в памяти:

	A[0][0]	A[0][1]	A[0][2]	A[1][0]	A[1][1]	A[1][2]
	A[0]			A[1]		
Адрес	n	n+4	n+8	n+12	n+16	n+20
Значение	a[0][0]	a[0][1]	a[0][2]	a[1][0]	a[1][1]	a[1][2]
Индекс	0 0	0 1	0 2	1 0	1 1	1 2

3 Инициализация массивов

```
int a1[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
int a2[10] = {0, 1, 2, 3, 4};
int a3[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
char str1[6] = "hello";
char str2 [ 6 ] = {'h', 'e', 'l', 'l', 'o', '\0'};
char str3[] = "hello";
```

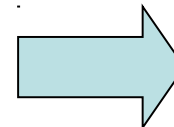
```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a[5] = {10, 11, 12, 13, 14};
6      printf("a = %d %d %d %d %d\n", a[0], a[1], a[2], a[3], a[4]);
7
8      float b[10] = {0, 1, 2, 3, 4};
9      printf("b = %f %f %f %f %f\n", b[0], b[2], b[4], b[6], b[8]);
10
11     float c[] = {10, 20, 30, 40, 50};
12     printf("c = %.1f %.1f %.1f %.1f %.1f\n", c[0], c[1], c[2], c[3], c[4]);
13
14     char str1[6] = "hello";
15     printf("str1 = %s\n", str1);
16     printf("str1[3] = %c\n", str1[3]);
17     char str2[6] = {'h', 'e', 'l', 'l', 'o', '\0'};
18     printf("str2 = %s\n", str2);
19     printf("str2[2] = %c\n", str2[2]);
20 }
21
```

```
a = 10 11 12 13 14
b = 0.000000 2.000000 4.000000 0.000000 0.000000
c = 10.0 20.0 30.0 40.0 50.0
str1 = hello
str1[3] = l
str2 = hello
str2[2] = l
```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Вариант 1:\n");
6      int a[2][3] = { 1, 2, 3, 4, 5, 6 };
7      printf("%d %d %d\n", a[0][0], a[0][1], a[0][2]);
8      printf("%d %d %d\n", a[1][0], a[1][1], a[1][2]);
9
10     printf("Вариант 2:\n");
11     int b[2][3] = { {10, 20, 30},
12                    {40, 50, 60} };
13     printf("%d %d %d\n", b[0][0], b[0][1], b[0][2]);
14     printf("%d %d %d\n", b[1][0], b[1][1], b[1][2]);
15
16
17     printf("Вариант 3:\n");
18     int c[][3] = { { 18, 21, 5 },
19                   { 6, 7 },
20                   { 30, 52, 34 } };
21     printf("%d %d %d\n", c[0][0], c[0][1], c[0][2]);
22     printf("%d %d %d\n", c[1][0], c[1][1], c[1][2]);
23     printf("%d %d %d\n", c[2][0], c[2][1], c[2][2]);
24
25     printf("Вариант 4:\n");
26     char s[][17] = { "Строка 1",
27                      "строка 2",
28                      "Строка 3" };
29     printf("%s\n%s\n%s\n", s[0], s[1], s[2]);
30
31     return 0;
32 }

```



```

Вариант 1:
1 2 3
4 5 6

Вариант 2:
10 20 30
40 50 60

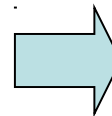
Вариант 3:
18 21 5
6 7 0
30 52 34

Вариант 4:
Строка 1
строка 2
Строка 3

```

3 Задание значений и вывод на экран массива

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a[5];
6      int i;
7
8      printf("Ввод элементов массива\n");
9      for (i = 0; i<5; i++)
10     {
11         printf("a[%d] = ", i);
12         scanf("%d", &a[i]);
13     }
14
15     printf("Вывод элементов массива\n");
16     for (i = 0; i<5; i++)
17     {
18         printf("%d ", a[i]);
19     }
20
21     return 0;
22 }
```

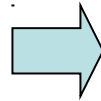


```
Ввод элементов массива
a[0] = 4
a[1] = -5
a[2] = 100
a[3] = 0
a[4] = -30
Вывод элементов массива
4 -5 100 0 -30
```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n=3, m=2;
6      float a[n][m];
7      int i,j;
8
9      printf("Ввод элементов массива\n");
10     for (i = 0; i<n; i++)
11     {
12         printf("Ввод строки %d:\n", i);
13         for (j = 0; j<m; j++)
14         {
15             printf("a[%d][%d] = ", i, j);
16             scanf("%f", &a[i][j]);
17         }
18     }
19
20     printf("Вывод элементов массива\n");
21     for (i = 0; i<n; i++)
22     {
23         for (j = 0; j<m; j++)
24         {
25             printf("%6.2f ", a[i][j]);
26         }
27         printf("\n");
28     }
29
30     return 0;
31 }

```



```

Ввод элементов массива
Ввод строки 0:
a[0][0] = 3
a[0][1] = -5.8
Ввод строки 1:
a[1][0] = -4.4
a[1][1] = 100
Ввод строки 2:
a[2][0] = 5
a[2][1] = -20
Вывод элементов массива
    3.00  -5.80
   -4.40 100.00
    5.00 -20.00

```

4 Строки

“Borland C++”



B	o	r	l	a	n	d		C	+	+	\0
---	---	---	---	---	---	---	--	---	---	---	----

```
char str[12]; // строка до 11 символов и последний '\0'
```

```

1  #include <stdio.h>
2
3  int main()
4  {
5      char str1[] = "Введите значение: ";
6      char str2[50];
7
8      puts(str1);
9      gets(str2);
10     puts(str2);
11
12     puts("-----");
13
14     printf("%s\n", str1);
15     scanf("%s", str2);
16     printf("%s", str2);
17
18     return 0;
19 }

```



```

Введите значение:
программирование на языке Си
программирование на языке Си
-----
Введите значение:
программирование на языке Си
программирование

```

Функции для обработки строк находятся в библиотеке string.h

Функция	Пояснение
<u>strlen</u> (имя_строки)	определяет длину указанной строки, без учёта нуля-символа
Копирование строк	
<u>strcpy</u> (s1,s2)	выполняет побайтное копирование символов из строки s2 в строку s1
<u>strncpy</u> (s1,s2, n)	выполняет побайтное копирование n символов из строки s2 в строку s1. Возвращает значения s1
Конкатенация строк	
<u>strcat</u> (s1,s2)	объединяет строку s2 со строкой s1. Результат сохраняется в s1
<u>strncat</u> (s1,s2,n)	объединяет n символов строки s2 со строкой s1. Результат сохраняется в s1
Сравнение строк	
<u>strcmp</u> (s1,s2)	сравнивает строку s1 со строкой s2 и возвращает результат типа int: 0 – если строки эквивалентны, >0 – если s1<s2, <0 — если s1>s2 (с учётом регистра)
<u>strncmp</u> (s1,s2,n)	сравнивает n символов строки s1 со строкой s2 и возвращает результат типа int: 0 –если строки эквивалентны, >0 – если s1<s2, <0 — если s1>s2 (с учётом регистра)
<u>stricmp</u> (s1,s2)	сравнивает строку s1 со строкой s2 и возвращает результат типа int : 0 –если строки эквивалентны, >0 – если s1<s2, <0 — если s1>s2 (без учёта регистра)
<u>strnicmp</u> (s1,s2,n)	сравнивает n символов строки s1 со строкой s2 и возвращает результат типа int: 0 –если строки эквивалентны, >0 – если s1<s2, <0 — если s1>s2 (без учёта регистра)

Обработка символов

<u>isalnum(c)</u>	возвращает значение true, если с является буквой или цифрой, и false в других случаях
<u>isalpha(c)</u>	возвращает значение true, если с является буквой, и false в других случаях
<u>isdigit(c)</u>	возвращает значение true, если с является цифрой, и false в других случаях
<u>islower(c)</u>	возвращает значение true, если с является буквой нижнего регистра, и false в других случаях
<u>isupper(c)</u>	возвращает значение true, если с является буквой верхнего регистра, и false в других случаях
<u>isspace(c)</u>	возвращает значение true, если с является пробелом, и false в других случаях
<u>toupper(c)</u>	если символ с, является символом нижнего регистра, то функция возвращает преобразованный символ с в верхнем регистре, иначе символ возвращается без изменений

Функции поиска

<u>strchr(s,c)</u>	поиск первого вхождения символа с в строке s. В случае удачного поиска возвращает указатель на место первого вхождения символа с. Если символ не найден, то возвращается ноль.
<u>strcspn(s1,s2)</u>	определяет длину начального сегмента строки s1, содержащего те символы, которые не входят в строку s2
<u>strspn(s1,s2)</u>	возвращает длину начального сегмента строки s1, содержащего только те символы, которые входят в строку s2
<u>strprbk(s1,s2)</u>	возвращает указатель первого вхождения любого символа строки s2 в строке s1

Функции преобразования

<u>atof(s1)</u>	преобразует строку s1 в тип double
<u>atoi(s1)</u>	преобразует строку s1 в тип int
<u>atol(s1)</u>	преобразует строку s1 в тип long int

4 Поиск в массиве

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int n=3,m=4;
7      int array[n][m];
8      unsigned i, j; int max;
9
10     for (i = 0; i < n; ++i)
11     {
12         for (j = 0; j < m; ++j)
13         {
14             array[i][j] = rand() % 40 + 20;
15             printf("%4d", array[i][j]);
16         }
17         printf("\n");
18     }
19
20     max = array[0][0];
21     for (i = 0; i < n; ++i)
22     {
23         for (j = 0; j < m; ++j)
24         {
25             if (array[i][j] > max)
26             {
27                 max = array[i][j];
28             }
29         }
30     }
31     printf("\nmax = %d\n", max);
32     return 0;
33 }
```



Линейный поиск
максимального
значения в матрице

43	26	37	55
53	35	46	32
29	41	22	47

max = 55

Бинарный поиск?

5 Сортировка в массиве

Метод «пузырька»

{1, 5, 2, 7, 6, 3}



1, 2, 5, 7, 6, 3

1, 2, 5, 6, 7, 3

1, 2, 5, 6, 3, 7



1, 2, 5, 3, 6, 7

1, 2, 3, 5, 6, 7

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int size = 6;
6      int i, j;
7      int tmp;
8
9      int a[] = {1, 5, 2, 7, 6, 3};
10
11     for (i = 1; i < size; i++)
12     {
13         for (j = 1; j < size; j++)
14         {
15             if (a[j] > a[j-1])
16             {
17                 tmp = a[j];
18                 a[j] = a[j-1];
19                 a[j-1] = tmp;
20             }
21         }
22     }
23
24     for (i = 0; i < size; i++)
25         printf("%d ", a[i]);
26 }

```

$(n-1)^2$ шагов



7 6 5 3 2 1

Какие ещё есть
алгоритмы
сортировки?